

Writing Documentation

Fahren Retreat 2026

Slides at <https://tinyurl.com/zmdty6xx>

The current landscape

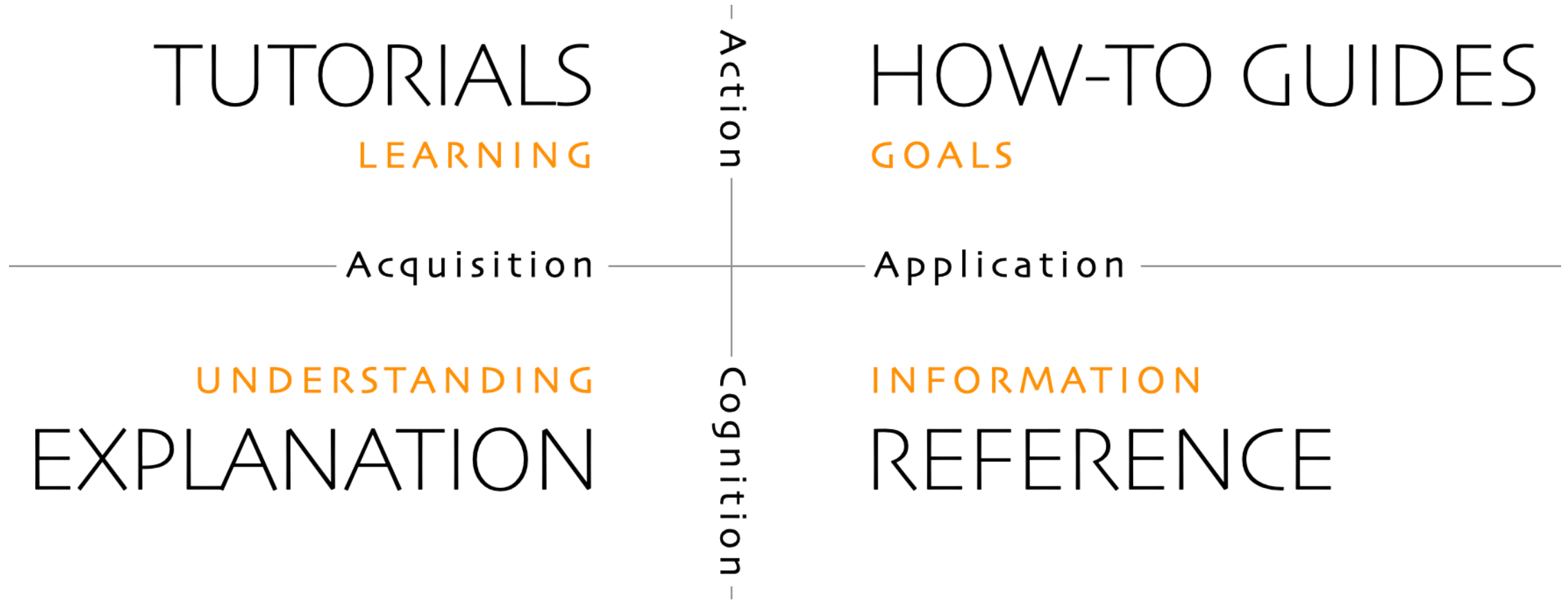
Working

- Nice NMMA documentation site set up
- In `--help` text 66/69 args are documented
- Existing docstrings follow good conventions

Needs work

- Of the CLI commands only 4/18 documented
- Only ~21% of public functions have docstrings
- `tutorials/README.md` currently empty

Diátaxis



Tutorials

A tutorial's purpose is to help the pupil acquire basic competence.

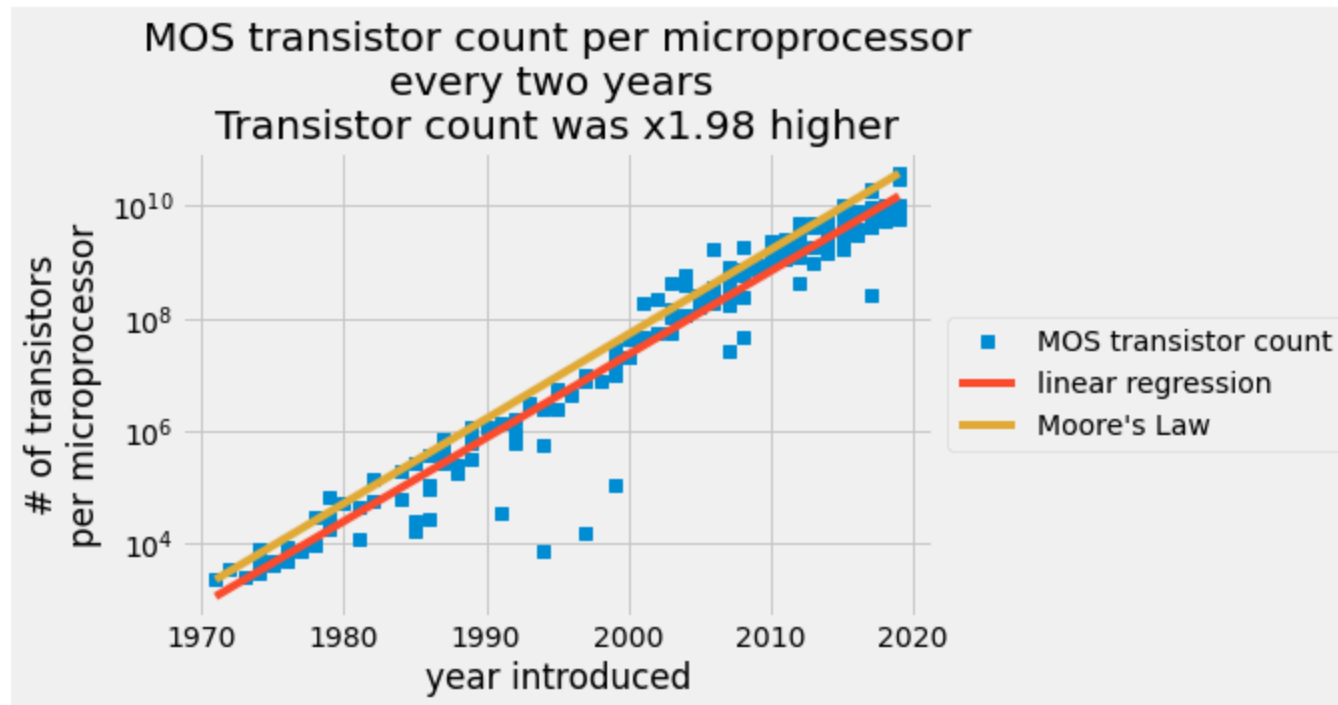
- **Provide a clear aim at the start:** This should be something they do, **not** a learning objective. *"In this tutorial we will set-up and run parameter estimation on a BBH injection."*
- **Deliver visible results early:** Understanding comes from connecting cause with effect, every step should produce a comprehensible result
- **Maintain a constant narrative:** Provide pointers throughout that they are on the right track, and what they should be learning from an action *"You will notice that..."*, *"After a few moments you will see..."*
- **Predict roadblocks:** If there are possible places things might go wrong or confuse the user, catch them early *"If the output doesn't show ..., you have probably forgotten to ... :*
"This command will probably return several hundred lines of logs in your terminal."

- **Encourage repetition:** Learning comes from repetition, how can the user repeat this tutorial in another way?
- **Minimise explanation:** Later, when the user is ready, they will seek explanation, but right now they are concerned with doing.
- **Ignore options and alternatives:** There may be many interesting diversions along the way, but this can be overwhelming for the user.
- **Use clear language:** *"First, do x. Now, do y. Now that you have done y, do z."*
- **At the end, summarise what's been achieved** *"You have now carried out your first parameter estimation run!"*

Example: Determining Moore's Law with real data in NumPy

Determining Moore's Law with real data in NumPy

Numpy Community



The number of transistors reported per a given chip plotted on a log scale in the y axis with the

How-to guides

A how-to guide's purpose is to help the already-competent user perform a particular task correctly.

- **Focus on a particular task or problem:** Assume the user knows what they want to achieve
- **Address real world complexity:** Make it general enough to be adaptable to real use-cases
- **Omit the unnecessary:** Practical usability is more helpful than completeness
- **Provide a set of instructions:** A how-to guide describes an executable solution to a real-world task in a logical order
- **Let it fork:** unlike a tutorial's single path, a how-to guide will typically fork and branch, describing different routes to the same destination.

Example: **How to index** `ndarrays`

How to index `ndarrays`

➞ See also

[Indexing on ndarrays](#)

This page tackles common examples. For an in-depth look into indexing, refer to [Indexing on ndarrays](#).

Access specific/arbitrary rows and columns

Use [Basic indexing](#) features like [Slicing and striding](#), and [Dimensional indexing tools](#).

```
>>> a = np.arange(30).reshape(2, 3, 5)
>>> a
array([[[ 0,  1,  2,  3,  4],
        [ 5,  6,  7,  8,  9],
        [10, 11, 12, 13, 14]],
       [[15, 16, 17, 18, 19],
```


What is the difference between tutorials and how-to guides?

Tutorial	How-to Guide
Helps the learner acquire basic competence .	Helps a competent user complete a particular task .
Provides a structured learning experience through hands-on practice.	Directs the user's work toward a successful result.
Follows a carefully managed, linear path with no alternatives.	Can branch into different routes depending on the situation.
Familiarises the learner with tools, processes, language, and behaviour.	Assumes the user is already familiar with these things.
Takes place in a controlled learning environment .	Applies to the unpredictable real world .
Eliminates unexpected situations .	Prepares the user to handle unexpected situations .
Must be safe and allow the learner to start again.	Cannot always guarantee safety ; mistakes may have consequences.
Responsibility lies primarily with the teacher .	Responsibility lies with the user .
Makes basic and embodied knowledge explicit .	Relies on this knowledge as implicit .
Uses concrete and specific tools, materials, and conditions.	Takes a general approach to accommodate different situations.
Teaches general skills and principles for future use.	Focuses on completing a particular task .

References

Reference guides are information-oriented, **technical descriptions** of the machinery and how to operate it. The most common example of references are API documentation derived from docstrings.

- **Code lead:** where tutorials and how to guides are led by the needs of the user, references are lead by the code
- **A full description:** Reference material is like a map, users won't read the full thing but find the technical details of what they needed to know
- **Adopt standard patterns:** Reference material is useful when it is consistent and easy to navigate
- **Provide examples:** An example of usage of a command can be a succinct way of illustrating it and its context

How to write a docstring

```
def function_name(arg, option=None):  
    """What the function does in one sentence.  
  
    Parameters  
    -----  
    arg : type  
        What it is and its units, if it's a physical quantity.  
    option : type, optional  
        Same, plus the default and when to override it.  
  
    Returns  
    -----  
    type  
        What comes back with units or shape if it's an array.  
  
    Examples  
    -----  
    One line example.  
    """
```

Example: `numpy.mean`

`numpy.mean`

`numpy.mean(a, axis=None, dtype=None, out=None, keepdims=<no value>, *, where=<no value>)` [\[source\]](#)

Compute the arithmetic mean along the specified axis.

Returns the average of the array elements. The average is taken over the flattened array by default, otherwise over the specified axis. `float64` intermediate and return values are used for integer inputs.

Parameters:

a : array_like

Array containing numbers whose mean is desired. If *a* is not an array, a conversion is attempted.

axis : None or int or tuple of ints, optional

Axis or axes along which the means are computed. The default is to compute the mean of the flattened array.

If this is a tuple of ints, a mean is performed over multiple axes, instead of a single axis or all the axes as before.

dtype : data-type, optional

Explanations

Explanation is an **understanding-oriented**, discursive treatment of a subject, that permits reflection.

- **Make connections:** When writing explanation you are helping to weave a web of understanding for your readers.
- **Provide context:** Provide background and context in your explanation, explaining why things are - design decisions, historical reasons, technical constraints - draw implications, mention specific examples.
- **Talk about the subject, not the task:** Explanation guides are about a topic in the sense that they are *around* it.
- **Insert your opinion:** As a developer, this is where you can share your perspective on why decisions were made.

Example: Broadcasting

Broadcasting

➔ See also

[numpy.broadcast](#)

The term broadcasting describes how NumPy treats arrays with different shapes during arithmetic operations. Subject to certain constraints, the smaller array is “broadcast” across the larger array so that they have compatible shapes. Broadcasting provides a means of vectorizing array operations so that looping occurs in C instead of Python. It does this without making needless copies of data and usually leads to efficient algorithm implementations. There are, however, cases where broadcasting is a bad idea because it leads to inefficient use of memory that slows computation.

NumPy operations are usually done on pairs of arrays on an element-by-element basis. In the simplest case, the two arrays must have exactly the same shape, as in the following example:

```
>>> import numpy as np
>>> a = np.array([1.0, 2.0, 3.0])
>>> b = np.array([2.0, 2.0, 2.0])
>>> a * b
array([2.  4.  6.])
```

How to generate documentation

- **Docstring:** the function itself, in `nmma/**/* .py`
- **How-to / explanation:** a new or existing page in `doc/* .md`
- **Tutorial:** a notebook in `tutorials/* .ipynb`

Preview pages before you open a PR:

```
pip install -e ".[doc]"  
cd doc && make html    # open _build/html/index.html
```

Warning: a new `doc/* .md` page has to be added to the toctree in `doc/index.rst`, or it's invisible on the built site.

Possible future work: wiring up autodoc so docstrings render as realAPI pages is possible, but not currently set up.

Activity

- Pairs: Person A and Person B
- Each person gets a function with **no docstring**, you will now write the docstring
- Swap **only the docstrings**, the other person will now write the function given the docstring
- Compare, run both functions on the same inputs and check you get the same outputs

Person A: Write the docstring for [treasure_trail.py](#)

Person B: Write the docstring for [available_books.py](#)

After the swap, test your written function with [treasure_trail_test.py](#) and [available_books_test.py](#)